

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.optimize import fsolve
4 from scipy.integrate import solve_ivp
5 from scipy.integrate import quad

```

```

1 '''
2 multitubular fixed bed reactor
3 we know feed and outlet and rate equations
4 find length, number of tubes
5 catalyst diameter between 2-5 mm
6 particle density is 2400 kg/m3, pore volume is 0.5 cm3/g, Sg = 3
7 check value of lambda (path length), check ra = 2Vg/Sg
8 equation for design -  $Udca/dz = (-ra)$  or  $W/Fa0 = \int dxa/(-ra)$ 
9 we know ra for all reactions
10 calculate W separately
11 '''
12

```

```

'\nmultitubular fixed bed reactor\nwe know feed and outlet and rate
equations\nfind length, number of tubes\ncatalyst diameter between
2-5 mm\nparticle density is 2400 kg/m3, pore volume is 0.5 cm3/g, S
g = 3.4 m2/g\ncheck value of lambda (path length) check ra = 2Vg/S

```

```

1 import numpy as np
2 from scipy.integrate import solve_ivp
3
4
5 class FlowCharacterstics:
6
7     def __init__(self):
8         # Operating conditions
9         self.P = 2.2e5 # Pa
10        self.T = 350 + 273.15 # K
11        self.R = 8.314
12
13        # Feed conditions
14        self.feed = {
15            'IB': {'kmol_h': 41.75},
16            'O2': {'kmol_h': 83.50},
17            'N2': {'kmol_h': 296.05},
18            'steam': {'kmol_h': 62.63}
19        }
20
21        self.FIB0 = self.feed['IB']['kmol_h']
22        self.FO20 = self.feed['O2']['kmol_h']
23        self.FN20 = self.feed['N2']['kmol_h']
24        self.Fsteam0 = self.feed['steam']['kmol_h']
25
26        # catalyst properties
27        self.dp_catalyst = 0.003 #m
28        self.rho_p = 2400 #kg/m3
29        self.Sg = 3.4

```

```

29     self.s_g = 0.1
30     self.V_g = 0.5
31
32     self.ra = (2 * self.V_g * 10**-6 / self.s_g)
33     print(f"pore radius is {self.ra} m")
34
35     self.d_ib = 4.8e-10
36     self.Kb = 1.38e-23
37     self.mean_lambda = self.Kb * self.T / (
38         np.sqrt(2) * np.pi * self.d_ib**2 * self.P
39     )
40
41     print(f"mean path length is {self.mean_lambda} m")
42
43     if self.mean_lambda > self.ra:
44         print("knudsen flow dominates")
45     else:
46         print("bulk flow dominates")
47
48 FlowCharacterstics()

```

```

pore radius is 2.941176470588235e-07 m
mean path length is 3.8185779431539756e-08 m
bulk flow dominates
<__main__.FlowCharacterstics at 0x78fa04992b40>

```

Species	Mole Balance
Isobutene (IB)	$\left(\frac{dF_{IB}}{dW} = -r_1 \right)$ $-r_4 - r_7$ $-r_8 - r_{11}$
Methacrolein (MAC)	$\left(\frac{dF_{MAC}}{dW} = r_{11} \right)$ $-r_2 - r_3$ $-r_5 - r_6$ $-r_9 - r_{10}$ $-r_{12}$
Methacrylic Acid (MAA)	$\left(\frac{dF_{MAA}}{dW} = r_{12} \right)$
Formaldehyde (CH ₂ O)	$\left(\frac{dF_{CH_2O}}{dW} = r_1 \right)$ $-r_{13} - r_{14}$
Acetone	$\left(\frac{dF_{Acetone}}{dW} = r_1 \right)$
Acrolein	$\left(\frac{dF_{Acrolein}}{dW} \right)$ $= r_4 + r_5$
Acetic Acid	$\left(\frac{dF_{AcOH}}{dW} = r_2 \right)$
Acrylic Acid	$\left(\frac{dF_{Acrylic}}{dW} = r_3 \right)$
Acetaldehyde	$\left(\frac{dF_{Acetaldehyde}}{dW} \right)$ $= r_6$
	$\left(\frac{dF_{O_2}}{dW} = -r_1 \right)$ $-3r_2 - 2r_3$

$$\begin{aligned} & -2.5r_4 \\ & -1.5r_5 \\ & -2.5r_6 \\ \text{Oxygen (O}_2\text{)} & -4r_7 - 2r_8 \\ & -5.5r_9 \\ & -3.5r_{10} \\ & -r_{11} \\ & -0.5r_{12} \\ & -r_{13} \\ & -0.5r_{14} \end{aligned}$$

$$\begin{aligned} & \left(\frac{dF_{CO}}{dW} = 4r_8 \right) \\ \text{Carbon Monoxide (CO)} & +4r_{10} \\ & +r_{14} \end{aligned}$$

$$\begin{aligned} & \left(\frac{dF_{CO_2}}{dW} = r_2 \right) \\ \text{Carbon Dioxide (CO}_2\text{)} & +r_4 + 2r_6 \\ & +4r_7 + 4r_9 \\ & +r_{13} \end{aligned}$$

$$\begin{aligned} & \left(\frac{dF_{H_2O}}{dW} = r_2 \right) \\ \text{Water (H}_2\text{O)} & +r_3 + 2r_4 \\ & +r_5 + r_6 \\ & +4r_7 + 4r_8 \\ & +3r_9 \\ & +3r_{10} \\ & +r_{11} + r_{13} \\ & +r_{14} \end{aligned}$$

```

1
2 class Reactor_design:
3     def __init__(self):
4         self.R = 8.314
5         self.T = 350 + 273.15
6         self.P = 1.2 * 10**5
7         self.rho_b = 2200
8
9         # STOICHIOMETRY
10        self.nr = 14
11        self.ns = 14
12        self.nu = np.zeros((self.nr, self.ns))
13
14        self.nu[0, [0, 10, 1, 13]] = [-1, -1, 1, 1]
15        self.nu[1, [1, 10, 3]] = [-1, -0.5, 1]
16        self.nu[2, [1, 10, 4, 12, 13]] = [-1, -3, 1, 2, 1]
17        self.nu[3, [1, 10, 9, 12, 13]] = [-1, -2, 1, 1, 1]
18        self.nu[4, [1, 10, 7, 12, 13]] = [-1, -1.5, 1, 1, 1]
19        self.nu[5, [1, 10, 6, 2, 13]] = [-1, -2, 1, 2, 1]
20        self.nu[6, [0, 10, 12, 13]] = [-1, -6, 4, 4]
21        self.nu[7, [0, 10, 2, 13]] = [-1, -4, 4, 4]
22        self.nu[8, [0, 10, 7, 12, 13]] = [-1, -2.5, 1, 1, 2]
23        self.nu[9, [1, 10, 12, 13]] = [-1, -5, 4, 3]
24        self.nu[10, [1, 10, 2, 13]] = [-1, -3, 4, 3]
25        self.nu[11, [0, 10, 5, 8]] = [-1, -1, 1, 1]

```

```

26     self.nu[12, [8, 10, 2, 13]] = [-1, -0.5, 1, 1]
27     self.nu[13, [8, 10, 12, 13]] = [-1, -1, 1, 1]
28
29     self.names = [
30         "IB", "MAC", "CO", "MAA", "AceticAcid", "Acetone",
31         "Acetaldehyde", "Acrolein", "Formaldehyde",
32         "AcrylicAcid", "O2", "N2", "CO2", "H2O", "FormicAcid",
33     ]
34
35     self.F_in = np.array([
36         41.75, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00, 0.00,
37         0.00, 83.50, 296.05, 0.00, 62.63, 0.00
38     ])
39
40     self.F_out = np.array([
41         0.17, 34.47, 5.12, 1.45, 1.45, 0.83, 1.45, 0.28, 0.00,
42         0.21, 26.35, 296.05, 7.56, 113.37, 0.00
43     ])
44
45     def flow_rates(self, xa):
46         FIB0 = self.F_in[0]
47         F020 = self.F_in[10]
48         FN20 = self.F_in[11]
49         FH200 = self.F_in[13]
50
51         FT0 = FIB0 + F020 + FN20 + FH200
52
53         self.y = np.array([0.83, 0.1, 0.03, 0.05])
54
55         FIB = FIB0 * (1 - xa)
56         FMAC = FIB0 * xa * self.y[0]
57         FS = FIB0 * xa * self.y[1]
58         FC0 = FIB0 * xa * self.y[2] * 4
59         FC02 = FIB0 * xa * self.y[3] * 4
60         F02 = F020 - (FIB0 * xa * self.y[0] + FIB0 * xa * self.y[1] +
61                     FIB0 * xa * self.y[2] + FIB0 * xa * self.y[3])
62         FN2 = FN20
63         FH20 = FH200 + FIB0 * xa * self.y[0] + FIB0 * xa * self.y[1] +
64             FIB0 * xa * self.y[2] * 4 + FIB0 * xa * self.y[3] * 4
65
66         FT = FIB + FMAC + FS + FC0 + FC02 + F02 + FN2 + FH20
67         flows = np.zeros(len(self.names))
68         flows[0] = FIB
69         flows[1] = FMAC
70         flows[2] = FC0
71         flows[3] = FS
72         flows[10] = F02
73         flows[11] = FN2
74         flows[12] = FC02
75         flows[13] = FH20
76
77         return FT, FT0, flows
78
79     def partial_pressures(self, xa):

```

```

80     FT, FT0, flows = self.flow_rates(xa)
81     P_total = self.P
82
83     partial_pressures = {}
84     mole_fractions = {}
85
86     for i, name in enumerate(self.names):
87         if FT > 0:
88             y_i = flows[i] / FT
89             mole_fractions[name] = y_i
90             partial_pressures[name] = y_i * P_total
91         else:
92             mole_fractions[name] = 0
93             partial_pressures[name] = 0
94
95     return partial_pressures, mole_fractions, FT
96
97 def concentrations_from_flows(self, xa):
98     FT, FT0, flows = self.flow_rates(xa)
99
100    R = self.R
101    T = self.T
102    P_total = self.P
103
104    concentrations = {}
105    mole_fractions = {}
106    partial_pressures = {}
107
108    for i, name in enumerate(self.names):
109        if FT > 0:
110            y_i = flows[i] / FT
111            mole_fractions[name] = y_i
112            partial_pressures[name] = y_i * P_total
113            concentrations[name] = (y_i * P_total) / (R * T)
114        else:
115            mole_fractions[name] = 0
116            partial_pressures[name] = 0
117            concentrations[name] = 0
118
119    return concentrations, partial_pressures, mole_fractions
120
121 def rate_IB_to_CO2(self, concentrations, xa):
122     C_IB = concentrations.get("IB", 0)
123     C_O2 = concentrations.get("O2", 0)
124
125     # SCALED DOWN significantly
126     A1 = 2.3e-03 # Changed from 2.3e04 to 2.3e-04
127     b1 = 0
128     Ea1 = 158155.2
129     k = A1 * self.T**b1 * np.exp(-Ea1 / (self.R * self.T))
130
131     rate1 = k * C_IB * (C_O2**0.5) / self.rho_b
132     return rate1
133

```

```

134     def rate_IB_to_CO(self, concentrations, xa):
135         C_IB = concentrations.get("IB", 0)
136         C_O2 = concentrations.get("O2", 0)
137
138         # SCALED DOWN significantly
139         A2 = 2.3e-03 # Changed from 2.3e04 to 2.3e-04
140         b2 = 0
141         Ea2 = 158155.2
142         k = A2 * self.T**b2 * np.exp(-Ea2 / (self.R * self.T))
143
144         rate2 = k * C_IB * (C_O2**0.5) / self.rho_b
145         return rate2
146
147     def rate_IB_to_MAC(self, partial_pressures, xa):
148         P_IB = partial_pressures.get("IB", 0) / 101325 # Conve
149         P_O2 = partial_pressures.get("O2", 0) / 101325
150
151         s = 3.4
152         s_kg = s * 1000
153
154         # SCALED DOWN significantly
155         k0_red = s_kg * 3.22e-11
156         k0_oxd = s_kg * 7.23e-12
157         Ea_oxd = -142e03
158         Ea_red = -52e03
159
160         kred = k0_red * np.exp(-Ea_red/(self.R * self.T))
161         koxd = k0_oxd * np.exp(-Ea_oxd/(self.R * self.T))
162         x = 0.8
163         y = 0.37
164         num = (kred * P_IB**x) * (koxd * P_O2**y)
165         den = kred * P_IB**x + koxd * P_O2**y
166
167         if den > 0:
168             rate3 = num / den
169         else:
170             rate3 = 0
171         return rate3
172
173     def rate_IB_to_side(self, concentrations, xa):
174         C_IB = concentrations.get("IB", 0)
175         C_O2 = concentrations.get("O2", 0)
176
177         # SCALED DOWN significantly
178         A = 1.7*10**-2.5 # Changed from 1.7e01 to 1.7e-03
179         b = 1.83
180         Ea = 30961.3
181         k = A * self.T**b * np.exp(-Ea / (self.R * self.T))
182
183         rate4 = k * C_IB * (C_O2**0.5) / self.rho_b
184         return rate4
185
186     def rA_total(self, xa):
187         concentrations, partial_pressures, mole_fractions, FT =

```

```

188
189     r1 = self.rate_IB_to_MAC(partial_pressures, xa)
190     r2 = self.rate_IB_to_C0(concentrations, xa)
191     r3 = self.rate_IB_to_C02(concentrations, xa)
192     r4 = self.rate_IB_to_side(concentrations, xa)
193
194     rA = r1 + r2 + r3 + r4
195     return rA
196
197 def dWdXa(self, xa, W):
198     FA0 = self.F_in[0] * 1000 / 3600 # mol/s
199     rA = self.rA_total(xa)
200
201     if rA > 1e-12:
202         return FA0 / rA
203     else:
204         return 1e10
205
206 def calculate_catalyst_weight(self, X_target, debug=False):
207     if debug:
208         print("\n" + "=" * 60)
209         print("DEBUG: Checking rates at X=0.5")
210         print("=" * 60)
211         conc, pp, mf, FT = self.concentrations_from_flows(0
212         print(f"C_IB = {conc.get('IB', 0):.2f} mol/m³")
213         print(f"C_O2 = {conc.get('O2', 0):.2f} mol/m³")
214         print(f"\nRate to MAC: {self.rate_IB_to_MAC(pp, 0.5
215         print(f"Rate to C0: {self.rate_IB_to_C0(conc, 0.5):
216         print(f"Rate to C02: {self.rate_IB_to_C02(conc, 0.5
217         print(f"Rate to side: {self.rate_IB_to_side(conc, 0
218         r_total = self.rA_total(0.5)
219         print(f"Total rate: {r_total:.6e} mol/(kg.s)")
220         FA0 = self.F_in[0] * 1000 / 3600
221         print(f"FA0: {FA0:.2f} mol/s")
222         print(f"dW/dX at X=0.5 = FA0/rA = {FA0/r_total:.2f}
223         print("=" * 60 + "\n")
224
225     def ode(xa, W):
226         return [self.dWdXa(xa, W[0])]
227
228     sol = solve_ivp(
229         ode,
230         [0, X_target],
231         [0],
232         method='RK45',
233         rtol=1e-6,
234         atol=1e-8,
235         max_step=0.01
236     )
237
238     if sol.success:
239         W_final = sol.y[0][-1]
240         return W_final, sol
241     else:

```

```

242         print("Integration failed!")
243         return 0, sol
244
245
246 # CREATE INSTANCE AND PRINT CONCENTRATIONS
247 reactor = Reactor_design()
248
249 # Test with conversion
250 xa_test = 0.994 # 99.4% conversion
251
252 # Get concentrations
253 concentrations, partial_pressures, mole_fractions, FT = reactor
254 FT, FT0, flows = reactor.flow_rates(xa_test)
255
256 # Get catalyst weight
257 W_cat, sol = reactor.calculate_catalyst_weight(xa_test)
258
259 # PRINT CONCENTRATIONS
260 print("=" * 90)
261 print("GAS PHASE CONCENTRATIONS")
262 print("=" * 90)
263 print(f"Temperature: {reactor.T:.2f} K ({reactor.T - 273.15:.2f} °C)")
264 print(f"Total Pressure: {reactor.P:.2f} Pa ({reactor.P/1e5:.2f} bar)")
265 print(f"Gas Constant: {reactor.R} J/(mol·K)")
266 print(f"Conversion: {xa_test*100:.1f}%")
267 print(f"Total Flow Rate: {FT:.2f} kmol/h")
268 print()
269
270 print("-" * 90)
271 print(f"{'Species':<18} {'Flow (kmol/h)':<15} {'Mole Fraction':<15}")
272 print(f"{'':<18} {'':<15} {'':<15} {'(Pa)':<18} {'(mol/m³)':<18}")
273 print("-" * 90)
274
275 for i, name in enumerate(reactor.names):
276     if flows[i] > 0 or mole_fractions[name] > 0:
277         C_value = concentrations[name]
278         print(f"{name:<18} {flows[i]:<15.4f} {mole_fractions[name]:<15.4f} ")
279         print(f"      {partial_pressures[name]:<18.2f} {C_value:<18.4f}")
280
281 print("-" * 90)
282
283 # CONCENTRATIONS IN DIFFERENT UNITS
284 print("\n" + "=" * 90)
285 print("CONCENTRATIONS IN DIFFERENT UNITS")
286 print("=" * 90)
287 print(f"{'Species':<18} {'mol/m³':<15} {'mol/L':<15} {'kmol/m³':<15}")
288 print("-" * 90)
289
290 for name, C in concentrations.items():
291     if C > 0:
292         C_mol_per_m3 = C
293         C_mol_per_L = C / 1000
294         C_kmol_per_m3 = C / 1000
295         C_kmol_per_L = C / 1e6

```



```

296         print(f"{name:<18} {C_mol_per_m3:<15.6f} {C_mol_per_L:<
297             f"{C_kmol_per_m3:<15.6f} {C_kmol_per_L:<15.6e}")
298
299 print("-" * 90)
300
301 # KEY COMPONENTS SUMMARY
302 print("\n" + "=" * 90)
303 print("KEY COMPONENTS CONCENTRATIONS")
304 print("=" * 90)
305 key_components = ["IB", "MAC", "O2", "N2", "CO2", "H2O", "CO"]
306
307 for comp in key_components:
308     if comp in concentrations:
309         idx = reactor.names.index(comp)
310         C_mol_per_m3 = concentrations[comp]
311         C_mol_per_L = C_mol_per_m3 / 1000
312         print(f"{comp:12} | Flow: {flows[idx]:6.2f} kmol/h | "
313             f"y = {mole_fractions[comp]:.4f} | "
314             f"P = {partial_pressures[comp]/1000:.2f} kPa | "
315             f"C = {C_mol_per_m3:.2f} mol/m³ ({C_mol_per_L:.4f}
316
317 # VERIFICATION - IDEAL GAS LAW
318 print("\n" + "=" * 90)
319 print("VERIFICATION - IDEAL GAS LAW")
320 print("=" * 90)
321 total_concentration_from_P = reactor.P / (reactor.R * reactor.T
322 sum_of_concentrations = sum(concentrations.values())
323 print(f"Total concentration from P/(RT): {total_concentration_f
324 print(f"Sum of individual concentrations: {sum_of_concentration
325 print(f"Match: {abs(total_concentration_from_P - sum_of_concent
326
327 # REACTION RATES
328 print("\n" + "=" * 90)
329 print("REACTION RATES")
330 print("=" * 90)
331
332 r1 = reactor.rate_IB_to_CO2(concentrations, xa_test)
333 r2 = reactor.rate_IB_to_CO(concentrations, xa_test)
334 r3 = reactor.rate_IB_to_MAC(partial_pressures, xa_test)
335 r4 = reactor.rate_IB_to_side(concentrations, xa_test)
336
337 print(f"Rate IB -> CO2: {r1:.6e} mol/(kg·s)")
338 print(f"Rate IB -> CO: {r2:.6e} mol/(kg·s)")
339 print(f"Rate IB -> MAC: {r3:.6e} mol/(kg·s)")
340 print(f"Rate IB -> side products: {r4:.6e} mol/(kg·s)")
341
342 total_rate = r1 + r2 + r3 + r4
343 print(f"\nTotal IB consumption rate: {total_rate:.6e} mol/(kg·s
344
345 # CATALYST WEIGHT CALCULATION
346 print("\n" + "=" * 90)
347 print("CATALYST WEIGHT CALCULATION")
348 print("=" * 90)
349
350

```

```

350 print(f"Target conversion: {xa_test*100:.1f}%")
351 print(f"Catalyst weight required: {W_cat:.2f} kg")
352
353 # Additional calculations
354 F_IB0 = reactor.F_in[0] # kmol/h
355 F_IB0_mol_s = F_IB0 * 1000 / 3600 # mol/s
356 print(f"IB inlet flow rate: {F_IB0:.2f} kmol/h ({F_IB0_mol_s:.2f} mol/s)")
357
358 # Space velocity calculation
359 if W_cat > 0:
360     WHSV = F_IB0 / W_cat
361     print(f"Weight Hourly Space Velocity (WHSV): {WHSV:.2f} h-1")
362 else:
363     print("Weight Hourly Space Velocity (WHSV): N/A (catalyst weight = 0)")
364
365 # Space time calculation
366 if W_cat > 0:
367     space_time = W_cat / F_IB0
368     print(f"Space Time: {space_time:.2f} h")
369
370 # SUMMARY
371 print("\n" + "=" * 90)
372 print("SUMMARY")
373 print("=" * 90)
374 print(f"Target conversion: {xa_test*100:.1f}%")
375 print(f"Catalyst weight required: {W_cat:.2f} kg")
376 print(f"IB feed rate: {F_IB0:.2f} kmol/h")
377 if W_cat > 0:
378     print(f"WHSV: {WHSV:.2f} h-1")
379 print(f"Total IB consumption rate at target conversion: {total_IB:.2f} kmol/h")
380
381 # Check if target conversion is achievable
382 if hasattr(sol, 't') and len(sol.t) > 0:
383     if sol.t[-1] < xa_test:
384         print(f"\n⚠ WARNING: Integration reached only {sol.t[-1]*100:.1f}% conversion")
385         print(f"Need more catalyst or check rate parameters")
386     else:
387         print(f"\n✓ Target conversion of {xa_test*100:.1f}% is achievable")

```

=====

GAS PHASE CONCENTRATIONS

=====

Temperature: 623.15 K (350.00 °C)
 Total Pressure: 120000.00 Pa (1.20 bar)
 Gas Constant: 8.314 J/(mol·K)
 Conversion: 99.4%
 Total Flow Rate: 487.66 kmol/h

Species	Flow (kmol/h)	Mole Fraction	Partial Pressure (Pa)
IB	0.2505	0.000514	61.64
MAC	34.4446	0.070632	8475.80
CO	4.9799	0.010212	1225.42
MAA	4.1499	0.008510	1021.18

O2	24.9857	0.051235	6148.25
N2	296.0500	0.607077	72849.20
CO2	8.2999	0.017020	2042.36
H2O	114.5044	0.234801	28176.16

CONCENTRATIONS IN DIFFERENT UNITS

Species	mol/m ³	mol/L	kmol/m ³	k
IB	0.011898	0.000012	0.000012	1
MAC	1.635980	0.001636	0.001636	1
CO	0.236527	0.000237	0.000237	2
MAA	0.197106	0.000197	0.000197	1
O2	1.186721	0.001187	0.001187	1
N2	14.061192	0.014061	0.014061	1
CO2	0.394212	0.000394	0.000394	3
H2O	5.438500	0.005439	0.005439	5

KEY COMPONENTS CONCENTRATIONS

IB	Flow: 0.25 kmol/h	y = 0.0005	P = 0.06 kPa	C =
MAC	Flow: 34.44 kmol/h	y = 0.0706	P = 8.48 kPa	C =
O2	Flow: 24.99 kmol/h	y = 0.0512	P = 6.15 kPa	C =
N2	Flow: 296.05 kmol/h	y = 0.6071	P = 72.85 kPa	C =
CO2	Flow: 8.30 kmol/h	y = 0.0170	P = 2.04 kPa	C =
H2O	Flow: 114.50 kmol/h	y = 0.2348	P = 28.18 kPa	C =
CO	Flow: 4.98 kmol/h	y = 0.0102	P = 1.23 kPa	C =

VERIFICATION - IDEAL GAS LAW

Total concentration from P/(RT): 23.162137 mol/m³
Sum of individual concentrations: 23.162137 mol/m³
Match: True

REACTION RATES

```

1 # TEST THE FUNCTIONS
2 if __name__ == "__main__":
3     reactor = Reactor_design()
4
5     # Test with conversion
6     xa_test = 0.994 # 99.4% conversion
7
8     # Get concentrations
9     concentrations, partial_pressures, mole_fractions, FT = re
10    FT, FT0, flows = reactor.flow_rates(xa_test)
11
12    # Get catalyst weight
13    W_cat, sol = reactor.calculate_catalyst_weight(xa_test)
14
15    # PRINT CONCENTRATIONS

```

```

16 print("=" * 90)
17 print("GAS PHASE CONCENTRATIONS")
18 print("=" * 90)
19 print(f"Temperature: {reactor.T:.2f} K ({reactor.T - 273.1
20 print(f"Total Pressure: {reactor.P:.2f} Pa ({reactor.P/1e5
21 print(f"Gas Constant: {reactor.R} J/(mol·K)")
22 print(f"Conversion: {xa_test*100:.1f}%")
23 print(f"Total Flow Rate: {FT:.2f} kmol/h")
24 print()
25
26 print("-" * 90)
27 print(f"{'Species':<18} {'Flow (kmol/h)':<15} {'Mole Fract
28 print(f"{'':<18} {'':<15} {'':<15} {'(Pa)':<18} {'(mol/m³)
29 print("-" * 90)
30
31 for i, name in enumerate(reactor.names):
32     if flows[i] > 0 or mole_fractions[name] > 0:
33         C_value = concentrations[name]
34         print(f"{name:<18} {flows[i]:<15.4f} {mole_fractio
35             f"{partial_pressures[name]:<18.2f} {C_value:
36
37 print("-" * 90)
38
39 # CONCENTRATIONS IN DIFFERENT UNITS
40 print("\n" + "=" * 90)
41 print("CONCENTRATIONS IN DIFFERENT UNITS")
42 print("=" * 90)
43 print(f"{'Species':<18} {'mol/m³':<15} {'mol/L':<15} {'kmol
44 print("-" * 90)
45
46 for name, C in concentrations.items():
47     if C > 0:
48         C_mol_per_m3 = C
49         C_mol_per_L = C / 1000
50         C_kmol_per_m3 = C / 1000
51         C_kmol_per_L = C / 1e6
52         print(f"{name:<18} {C_mol_per_m3:<15.6f} {C_mol_pe
53             f"{C_kmol_per_m3:<15.6f} {C_kmol_per_L:<15.6
54
55 print("-" * 90)
56
57 # KEY COMPONENTS SUMMARY
58 print("\n" + "=" * 90)
59 print("KEY COMPONENTS CONCENTRATIONS")
60 print("=" * 90)
61 key_components = ["IB", "MAC", "O2", "N2", "CO2", "H2O", "
62
63 for comp in key_components:
64     if comp in concentrations:
65         idx = reactor.names.index(comp)
66         C_mol_per_m3 = concentrations[comp]
67         C_mol_per_L = C_mol_per_m3 / 1000
68         print(f"{comp:12} | Flow: {flows[idx]:6.2f} kmol/h
69             f"y = {mole_fractions[comp]:.4f} | "
70             f"p = {partial_pressures[comp]:6.2f} Pa | C =

```

```

70         T"P = {partial_pressures[comp]/1000:.2f} kPa
71         f"C = {C_mol_per_m3:.2f} mol/m³ ({C_mol_per_
72
73     # VERIFICATION - IDEAL GAS LAW
74     print("\n" + "=" * 90)
75     print("VERIFICATION - IDEAL GAS LAW")
76     print("=" * 90)
77     total_concentration_from_P = reactor.P / (reactor.R * reac
78     sum_of_concentrations = sum(concentrations.values())
79     print(f"Total concentration from P/(RT): {total_concentrat
80     print(f"Sum of individual concentrations: {sum_of_concentr
81     print(f"Match: {abs(total_concentration_from_P - sum_of_cc
82
83     # REACTION RATES
84     print("\n" + "=" * 90)
85     print("REACTION RATES")
86     print("=" * 90)
87
88     r1 = reactor.rate_IB_to_CO2(concentrations, xa_test)
89     r2 = reactor.rate_IB_to_CO(concentrations, xa_test)
90     r3 = reactor.rate_IB_to_MAC(partial_pressures, xa_test)
91     r4 = reactor.rate_IB_to_side(concentrations, xa_test)
92
93     print(f"Rate IB -> CO2: {r1:.6e} mol/(kg.s)")
94     print(f"Rate IB -> CO: {r2:.6e} mol/(kg.s)")
95     print(f"Rate IB -> MAC: {r3:.6e} mol/(kg.s)")
96     print(f"Rate IB -> side products: {r4:.6e} mol/(kg.s)")
97
98     total_rate = r1 + r2 + r3 + r4
99     print(f"\nTotal IB consumption rate: {total_rate:.6e} mol/
100
101     # CATALYST WEIGHT CALCULATION
102     print("\n" + "=" * 90)
103     print("CATALYST WEIGHT CALCULATION")
104     print("=" * 90)
105
106     print(f"Target conversion: {xa_test*100:.1f}%")
107     print(f"Catalyst weight required: {W_cat:.2f} kg")
108
109     # Additional calculations
110     F_IB0 = reactor.F_in[0] # kmol/h
111     F_IB0_mol_s = F_IB0 * 1000 / 3600 # mol/s
112     print(f"IB inlet flow rate: {F_IB0:.2f} kmol/h ({F_IB0_mol
113
114     # Space velocity calculation
115     if W_cat > 0:
116         WHSV = F_IB0 / W_cat
117         print(f"Weight Hourly Space Velocity (WHSV): {WHSV:.2f
118     else:
119         print("Weight Hourly Space Velocity (WHSV): N/A (catal
120
121     # Space time calculation
122     if W_cat > 0:
123         space_time = W_cat / F_IB0
124         print(f"Space Time: {space_time:.2f} h")

```

```

124         print(f"Space time: {space_time:.2f} min")
125
126     # SUMMARY
127     print("\n" + "=" * 90)
128     print("SUMMARY")
129     print("=" * 90)
130     print(f"Target conversion: {xa_test*100:.1f}%")
131     print(f"Catalyst weight required: {W_cat:.2f} kg")
132     print(f"IB feed rate: {F_IB0:.2f} kmol/h")
133     if W_cat > 0:
134         print(f"WHSV: {WHSV:.2f} h-1")
135     print(f"Total IB consumption rate at target conversion: {t
136
137     # Check if target conversion is achievable
138     if hasattr(sol, 't') and len(sol.t) > 0:
139         if sol.t[-1] < xa_test:
140             print(f"\n⚠ WARNING: Integration reached only {s
141             print(f"    Need more catalyst or check rate parame
142         else:
143             print(f"\n✓ Target conversion of {xa_test*100:.1f}
144
145
146     # Print all reaction enthalpies at current temperature
147     reactor.print_all_reaction_enthalpies()
148
149     print("\n" + "=" * 100)
150     print("DETAILED REACTION INFORMATION")
151     print("=" * 100)
152
153     reactions_to_check = [0, 6, 7, 8, 11]
154     for rxn in reactions_to_check:
155         info = reactor.get_reaction_info(rxn)
156         print(f"\nReaction {rxn}: {info['reaction_name']}")
157         print(f"    Temperature: {info['temperature_C']:.2f}")
158         print(f"    Total ΔH: {info['delta_H_total_kJ_per_mol']:.2f}")
159         if 'delta_H_kJ_per_mol_organic' in info:
160             print(f"    Per mole of organic product: {info['
161         if 'delta_H_kJ_per_mol_CO2' in info:
162             print(f"    Per mole of CO2: {info['delta_H_kJ_p
163
164     print("\n" + "=" * 100)
165     print("TEMPERATURE SWEEP: IB → CO2 (Reaction 6)")
166     print("=" * 100)
167     T_range = np.linspace(298.15, 673.15, 10)
168     for T in T_range:
169         delta_H = reactor.heat_of_reaction(6, T, basis="mc
170         print(f"T = {T:.2f} K ({T-273.15:.2f} °C): ΔH = {d
171
172
173
174     # ADIABATIC TEMPERATURE RISE
175     print("\n" + "=" * 90)
176     print("ADIABATIC TEMPERATURE RISE")
177     print("=" * 90)
178

```

```
178
179     dT, T_ad, Q, Cp = reactor.adiabatic_temperature_rise()
180
181     print(f"Inlet Temperature: {reactor.T:.2f} K ({reactor.T -
182     print(f"Adiabatic Temperature Rise ( $\Delta T$ ): {dT:.2f} K")
183     print(f"Adiabatic Outlet Temperature: {T_ad:.2f} K ({T_ad
184     print(f"Total Heat Released: {Q:.2f} kJ/h")
185     print(f"Mixture Heat Capacity: {Cp:.2f} kJ/K·h")
```

NameError Traceback (most recent
call last)

/tmp/ipykernel_1710/1736120415.py in <cell line: 0>()

```
1 # TEST THE FUNCTIONS
2 if __name__ == "__main__":
----> 3     reactor = Reactor_design()
4
5     # Test with conversion
```

/tmp/ipykernel_1710/3152248302.py in __init__(self)

```
9         self.nr = 14
10         self.ns = 14
---> 11         self.nu = np.zeros((self.nr, self.ns))
12
13         self.nu[0, [0, 10, 1, 13]] = [-1, -1, 1, 1]
```

NameError: name 'np' is not defined

Next steps: [Explain error](#)